# lab4 report：递归求最终存款

**姓名**: 黄思睿 **日期**: 2024年12月14日

---

## 1. Purpose

本实验的目标是通过 LC-3 汇编语言实现递归算法，计算 Sakiko酱 在若干个月后的最终存款余额。递归算法需要遵循以下规则：

1. 每月收入翻倍，初始收入为 6。
2. 每月支出翻四倍，初始支出为 2。当支出超过收入时，支出重置为 2。
3. 计算从当前月数递减到 0 时的最终存款，并将结果存储到内存地址 `RESULT` (x3200)。

---

## 2. Principles

本实验通过递归公式实现以下存款更新规则：

1. **当月存款计算**：

$$S(i) = S(i-1) + Income(i) - Expense(i) \tag{1}$$

2. **收入规则**：收入每月翻倍。
3. **支出规则**：支出每月翻四倍，若超过收入，则重置为 2。

## 3. Procedure

该程序的目标是递归计算某账户在特定月份后的最终存款余额。每月收入翻倍，支出规则根据当月的情况调整。当收入不足以支付支出时，将下个月的支出重置为特定值。

核心递归思想如下：

1. 如果月份为零，直接返回当前存款作为结果。
2. 更新当月的存款余额，收入增加，支出减少。
3. 根据支出是否超出收入调整下个月的支出规则。
4. 减少月份计数后递归调用自身。
5. 递归完成后逐层返回并恢复之前的状态。

### 3.1. 实现该算法的关键代码

以下是实现算法的关键代码段：

- **递归基础条件检查与返回**：

```
ADD R1, R1, #0
BRz DONE ; 若 R1 == 0，则跳转到 DONE
```

- **递归调用与堆栈管理**：

```
STR R7, R6, #0 ; 保存 R7
ADD R6, R6, #-1 ; 堆栈指针向下移动
STR R1, R6, #0 ; 保存 R1
ADD R6, R6, #-1
...
JSR CALCULATE_SAVINGS ; 递归调用下一月
```

- **支出调整逻辑**：

```
NOT R5, R3 ; R5 = -收入
ADD R5, R5, #1
ADD R5, R5, R4 ; R5 = 支出 - 收入
BRzp RESET ; 若支出 >= 收入，重置
ADD R4, R4, R4
ADD R4, R4, R4 ; 下个月支出翻四倍
BRnzp CONTINUE
RESET
LD R4, RESET_EXP ; 支出重置为 2
```

---

### 3.2 具体实现思路

1. **递归调用的实现**：
   - 使用 `JSR` 指令递归调用函数 `CALCULATE_SAVINGS`。
   - 在递归调用中，使用堆栈保存当前寄存器的值，以确保返回时状态正确。
2. **关键代码分析**：
   - **递归停止条件**：检查月份是否为 0，若是则直接返回存款结果：

     ```
     ADD R1, R1, #0
     BRz DONE          ; 若 R1 == 0，则跳转到 DONE
     ```

   - **支出重置逻辑**：当支出大于等于收入时，将下个月的支出重置为 2：

```
NOT R5, R3          ; R5 = -收入
ADD R5, R5, #1
ADD R5, R5, R4      ; R5 = 支出 - 收入
BRzp RESET          ; 若支出 >= 收入，重置
```

- **堆栈管理**：保存并恢复寄存器状态：

```
STR R1, R6, #0      ; 保存当前月数
ADD R6, R6, #-1
LDR R1, R6, #0      ; 恢复月数
ADD R6, R6, #1
```

3. **调试过程**：
   - 初始阶段，程序未正确处理递归返回值，发现是因为未正确保存 `R7`。
   - 测试不同初始条件后发现逻辑错误：未处理 `Income` 和 `Expense` 溢出问题，修改后确保在每次更新支出和收入时避免负值。
   - **边界条件问题**：
     - 忘记在月份为零时停止递归。
     - 解决方案：在递归函数开头增加条件检查并跳转到结束逻辑。

---

## 3.4 全部代码

```
.ORIG x3000
; 初始化寄存器和变量
LDI R1, INPUT           ; 月数存储到 R1
LD R2, INITIAL_SAV    ; 初始存款存储到 R2
LD R3, INCOME         ; 初始收入存储到 R3
LD R4, EXPENSE        ; 初始支出存储到 R4
LD R6, STACK

; 调用递归函数计算最终存款
JSR CALCULATE_SAVINGS

; 将结果存储到 RESULT
HALT

; 递归函数 CALCULATE_SAVINGS
CALCULATE_SAVINGS
    ; 保存当前状态到堆栈
    STR R7, R6, #0       ;保存R7的值，以便后续函数一层一层返回时可以从栈中取出
    ADD R6, R6, #-1    ; 堆栈指针向下移动
    STR R1, R6, #0     ; 保存 R1（当前月数）
    ADD R6, R6, #-1
    STR R2, R6, #0     ; 保存 R2（当前存款）
    ADD R6, R6, #-1
```

```
    STR R3, R6, #0     ; 保存 R3（当前收入）
    ADD R6, R6, #-1
    STR R4, R6, #0     ; 保存 R4（当前支出）

    ; 如果月份为 0，则返回当前存款
    ADD R1, R1, #0
    BRz DONE           ; 若 R1 == 0，则跳转到 DONE

    ; 更新当前存款
    ADD R2, R2, R3     ; 存款加上收入
    NOT R5, R4         ; R5 = -支出
    ADD R5, R5, #1
    ADD R2, R2, R5     ; 存款减去支出

    ; 检查本月支出是否超过收入
    NOT R5, R3         ; R5 = -收入
    ADD R5, R5, #1
    ADD R5, R5, R4     ; R5 = 支出 - 收入

    BRzp RESET          ; 若支出 >= 收入，重置
    ADD R4, R4, R4
    ADD R4, R4, R4     ; 下个月支出翻四倍
    BRnzp CONTINUE
RESET
    LD R4, RESET_EXP   ; 下个月支出重置支出为 2

CONTINUE
    ; 更新下个月的收入
    ADD R3, R3, R3     ; 收入翻倍
    ADD R1, R1, #-1    ; 月份减 1
    ; 递归调用下一月
    JSR CALCULATE_SAVINGS

RESTORE
    ; 恢复堆栈中的状态
    LDR R4, R6, #0     ; 恢复支出
    ADD R6, R6, #1
    LDR R3, R6, #0     ; 恢复收入
    ADD R6, R6, #1
    LDR R2, R6, #0     ; 恢复存款
    ADD R6, R6, #1
    LDR R1, R6, #0     ; 恢复月数
    ADD R6, R6, #1
    LDR R7, R6, #0
    RET

DONE
    STI R2, RESULT      ; 将 R2（最终存款）存储到 RESULT
    BRnzp RESTORE
```

```
; 数据段
INITIAL_SAV .FILL #10 ; 初始存款为 10
INCOME      .FILL #6  ; 初始收入为 6
EXPENSE     .FILL #2  ; 初始支出为 2
RESET_EXP   .FILL #2  ; 支出重置为 2
INPUT       .FILL x3100 ; 输入的月份存储地址
RESULT      .FILL x3200 ; 最终结果存储地址
STACK       .FILL x6000

.END
```

---

### 3.5 Run Results

以 `INPUT` 中的月份为 3 运行程序，测试初始存款为 10，初始收入为 6，初始支出为 2，运行后结果存储在 `RESULT` 中，最终存款为 **66**。

汇编评测

6 / 6 个通过测试用例

- 平均指令数: 288.8333333333333
- 通过 5, 指令数: 202, 输出: 144
- 通过 6, 指令数: 237, 输出: 304
- 通过 7, 指令数: 272, 输出: 560
- 通过 8, 指令数: 307, 输出: 816
- 通过 9, 指令数: 340, 输出: 304
- 通过 10, 指令数: 375, 输出: 3374

---

## Traps and Pitfalls

1. **递归深度问题**：堆栈指针在每次调用时递减，容易导致溢出。
2. **逻辑边界条件**：支出和收入更新时需确保结果在合法范围内。

---

## Suggestions

1. 使用迭代方法代替递归可显著减少堆栈开销，提高程序效率。
2. 引入更多的中间检查指令以捕获溢出或无效结果。

---

## Discussion Questions

### 为什么迭代方法更高效?

- **内存占用**：递归方法每次调用需要保存上下文到堆栈，消耗额外内存，尤其当月份较大时，堆栈可能溢出。
- **指令数量**：递归方法增加了函数调用与返回的额外指令，而迭代方法直接在循环中更新变量，省去了保存与恢复状态的操作。

### 如何提高程序效率?

1. **改为迭代实现**：用循环实现递归逻辑，仅需要更新当前存款、收入和支出值。
2. **优化分支条件**：减少冗余检查与分支操作。（虽然我不会，嘻嘻）

# GO！还在GO！